

Integer Arithmetic

Chapter 7

Shift and Rotate Instructions

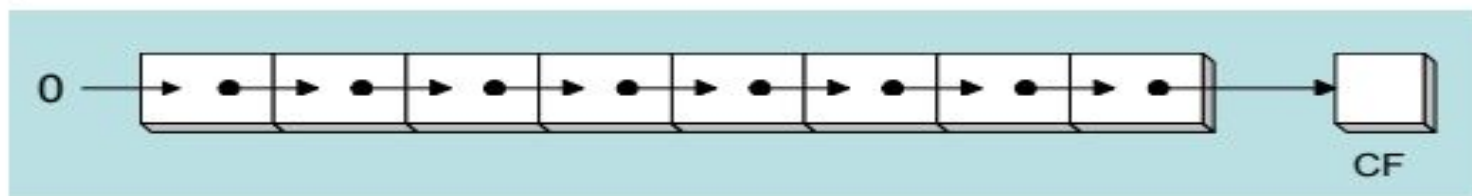
- Logical vs Arithmetic Shifts
- SHL Instruction
- SHR Instruction
- SAL and SAR Instructions
- ROL Instruction
- ROR Instruction
- RCL and RCR Instructions
- SHLD/SHRD Instructions

- Bit manipulation is an intrinsic part of computer graphics, data encryption, and hardware manipulation.
- *Bit shifting* means to move bits right and left inside an operand.
- Shift and Rotate instructions affect the overflow and carry flags.

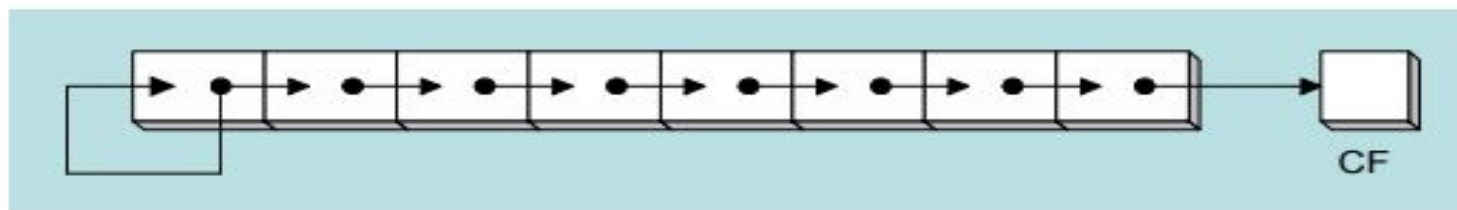


Logical vs arithmetic shifts

- A logical shift fills the newly created bit position with zero:



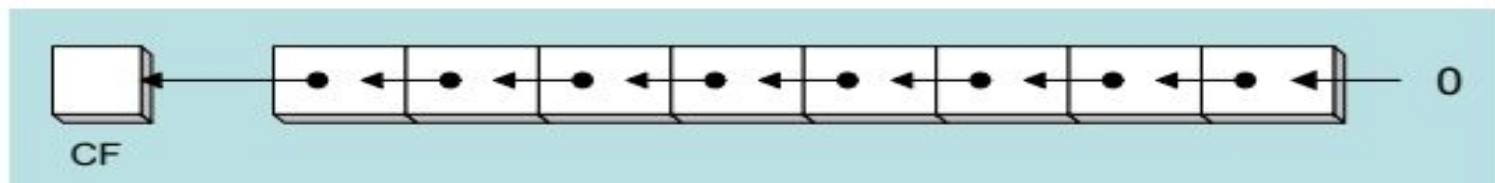
- An arithmetic shift fills the newly created bit position with a copy of the number's sign bit:



SHL instruction



- The SHL (shift left) instruction performs a logical left shift on the destination operand, filling the lowest bit with 0.



- Operand types: SHL *destination*, *count*

SHL *reg*, *imm8*

SHL *mem*, *imm8*

SHL *reg*, CL

SHL *mem*, CL

```
mov  bl,8Fh           ; BL = 10001111b
shl  bl,1             ; CF = 1, BL = 00011110b

mov  al,100000000b
shl  al,2             ; CF = 0, AL = 00000000b
```

Fast multiplication

Shifting left 1 bit multiplies a number by 2

```
mov dl,5  
shl dl,1
```

Before:

0	0	0	0	0	1	0	1
---	---	---	---	---	---	---	---

 = 5
After:

0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---

 = 10

Shifting left n bits multiplies the operand by 2^n

For example, $5 * 2^2 = 20$

```
mov dl,5  
shl dl,2  
; DL = 20
```

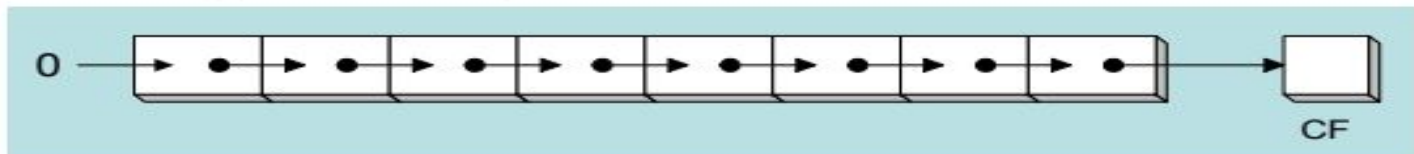
- SHL can perform multiplication by powers of 2.
- Shifting any operand left by n bits multiplies the operand by 2^n .
- For example, shifting the integer 5 left by 1 bit yields the product of $5 \times 2^1 = 10$.
- If binary 00001010 (decimal 10) is shifted left by two bits, the result is the same as multiplying 10 by 2^2

<code>mov dl,10</code>	<code>; before: 00001010</code>
<code>shl dl,2</code>	<code>; after: 00101000</code>

SHR instruction



- The SHR (shift right) instruction performs a logical right shift on the destination operand. The highest bit position is filled with a zero.



Shifting right n bits divides the operand by 2^n

```
mov  dl,80
shr  dl,1           ; DL = 40
shr  dl,2           ; DL = 10
```

```
mov  al,0D0h        ; AL = 11010000b
shr  al,1            ; AL = 01101000b, CF = 0
```

```
mov  al,00000010b
shr  al,2            ; AL = 00000000b, CF = 1
```

- *Bitwise division* is accomplished when you shift a number's bits in a rightward direction (toward the LSB).
- Shifting an unsigned integer right by n bits divides the operand by 2^n . In the following statements, we divide 32 by 2^1 , producing 16:

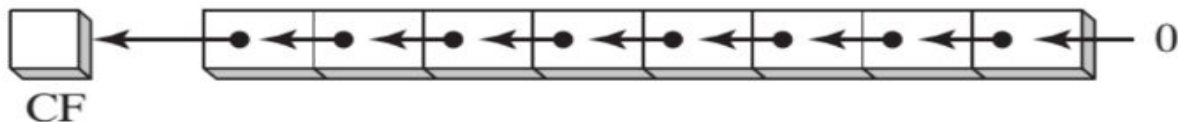
<code>mov dl,32</code>	Before:	<table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	1	0	0	0	0	0	= 32
0	0	1	0	0	0	0	0				
<code>shr dl,1</code>	After:	<table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	1	0	0	0	0	= 16
0	0	0	1	0	0	0	0				

<code>mov al,01000000b</code>	<code>; AL = 64</code>
<code>shr al,3</code>	<code>; divide by 8, AL = 00001000b</code>

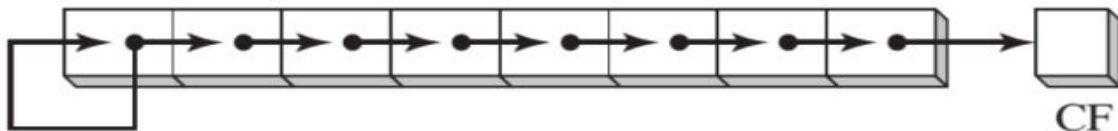
SAL and SAR instructions



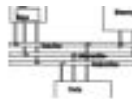
- The **SAL (shift arithmetic left)** Instruction works the same as the SHL instruction.



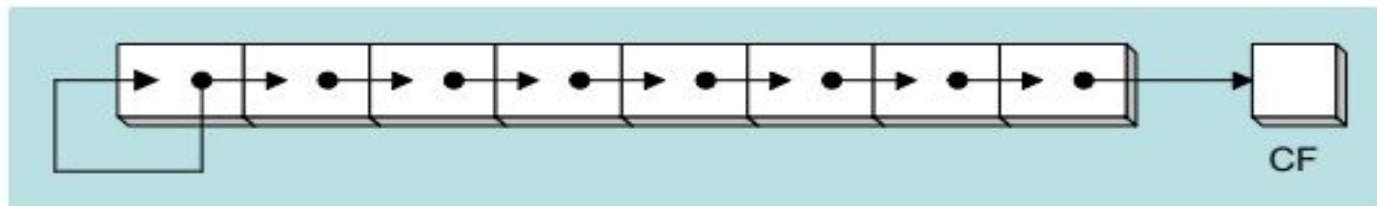
- The **SAR (shift arithmetic right)** instruction performs a right arithmetic shift on its destination operand:



SAL and SAR instructions



- SAL (shift arithmetic left) is identical to SHL.
- SAR (shift arithmetic right) performs a right arithmetic shift on the destination operand.



- An arithmetic shift preserves the number's sign.

```
mov dl, -80
sar dl, 1      ; DL = -40
sar dl, 2      ; DL = -10
```

- The operands for SAL and SAR are identical to those for SHL and SHR.

```
mov al,0F0h           ; AL = 11110000b (-16)
sar al,1              ; AL = 11111000b (-8), CF = 0
```

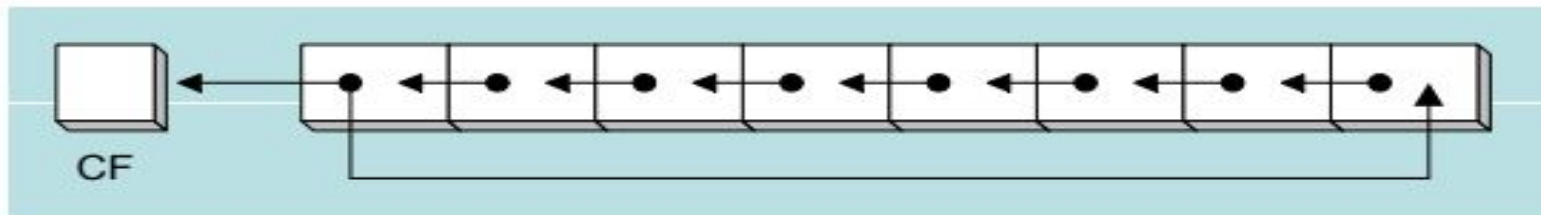
- You can divide a signed operand by a power of 2

```
mov dl,-128           ; DL = 10000000b
sar dl,3              ; DL = 11110000b (-16)
```



ROL instruction

- ROL (rotate) shifts each bit to the left
- The highest bit is copied into both the Carry flag and into the lowest bit
- No bits are lost



```
mov al,11110000b
```

```
rol al,1 ; AL = 11100001b
```

```
mov dl,3Fh
```

```
rol dl,4 ; DL = F3h
```

```
mov al,40h           ; AL = 010000000b
rol al,1             ; AL = 100000000b, CF = 0
rol al,1             ; AL = 000000001b, CF = 1
rol al,1             ; AL = 000000010b, CF = 0
```

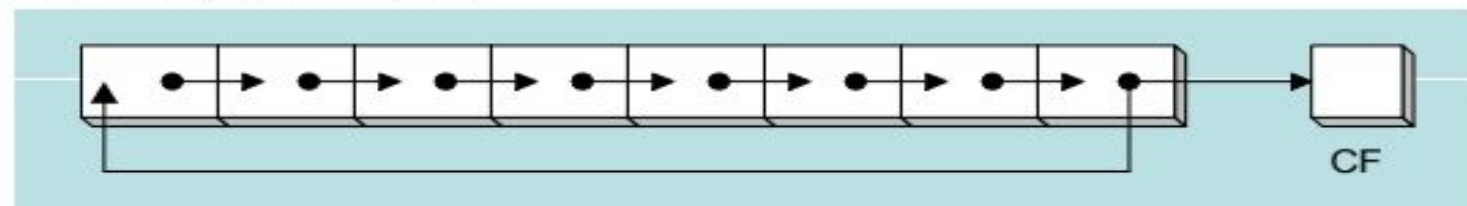
- When using a rotation count greater than 1, the Carry flag contains the last bit rotated out of the MSB position

```
mov al,001000000b
rol al,3             ; CF = 1, AL = 000000001b
```

ROR instruction



- ROR (rotate right) shifts each bit to the right
- The lowest bit is copied into both the Carry flag and into the highest bit
- No bits are lost



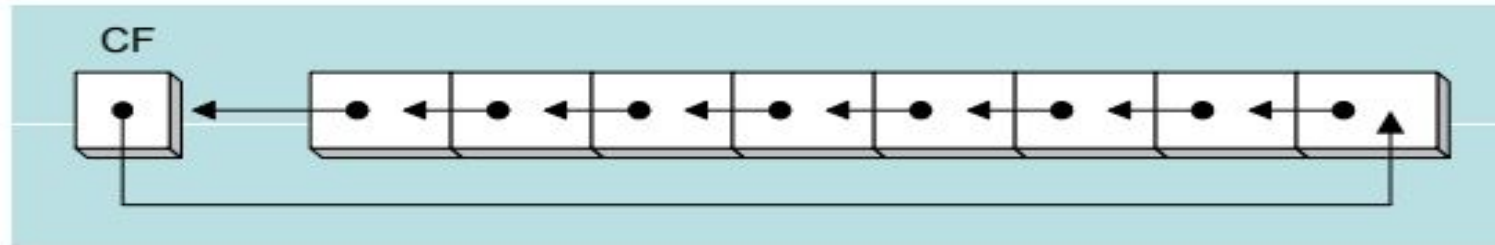
```
mov al,11110000b
ror al,1                ; AL = 01111000b

mov dl,3Fh
ror dl,4                ; DL = F3h
```




RCL instruction

- RCL (rotate carry left) shifts each bit to the left
- Copies the Carry flag to the least significant bit
- Copies the most significant bit to the Carry flag

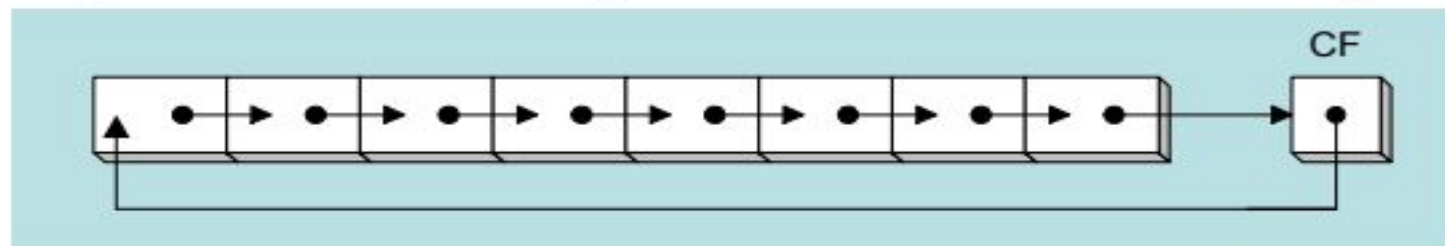


<code>clc</code>	<code>; CF = 0</code>
<code>mov bl, 88h</code>	<code>; CF, BL = 0 10001000b</code>
<code>rcl bl, 1</code>	<code>; CF, BL = 1 00010000b</code>
<code>rcl bl, 1</code>	<code>; CF, BL = 0 001000001b</code>

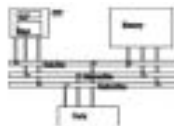


RCR instruction

- RCR (rotate carry right) shifts each bit to the right
- Copies the Carry flag to the most significant bit
- Copies the least significant bit to the Carry flag



```
stc                ; CF = 1
mov ah,10h         ; CF,AH = 00010000 1
rcr ah,1           ; CF,AH = 10001000 0
```

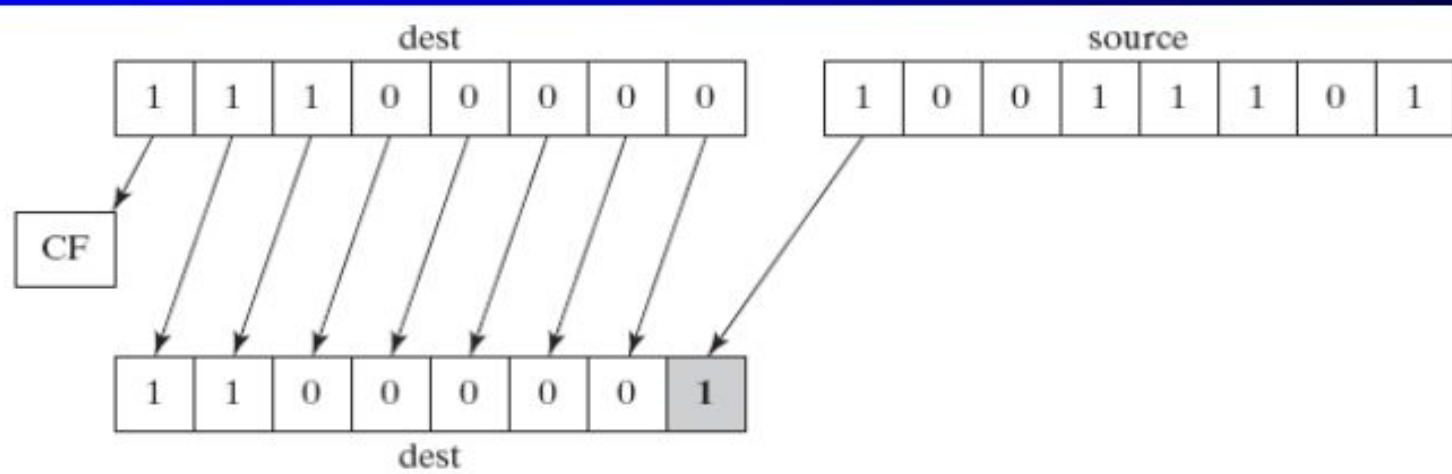


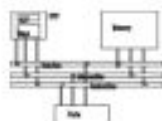
SHLD instruction

- Syntax: (shift left double)
SHLD destination, source, count
- Shifts a destination operand a given number of bits to the left
- The bit positions opened up by the shift are filled by the most significant bits of the source operand
- The source operand is not affected

Shift count of 1:

```
mov al,11100000b  
mov bl,10011101b  
shld al,bl,1
```





SHLD example

Shift **wval** 4 bits to the left and replace its lowest 4 bits with the high 4 bits of **AX**:

```
.data
wval WORD 9BA6h
.code
mov ax, 0AC36h
shld wval, ax, 4
```

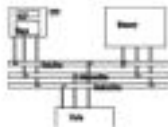
Before:



After:



SHRD instruction



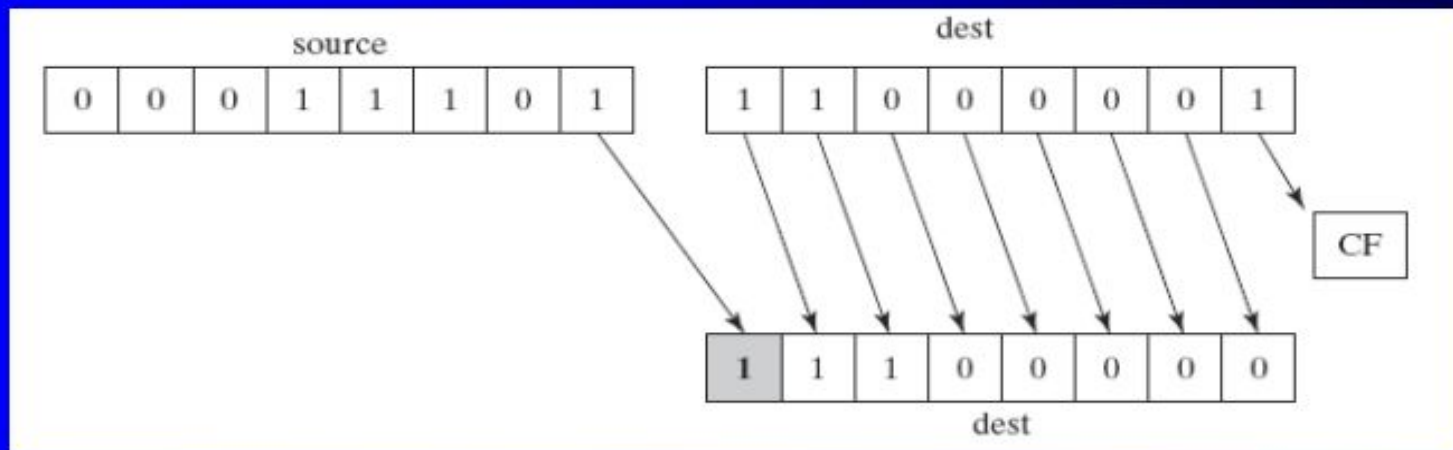
- Syntax:
SHRD destination, source, count
- Shifts a destination operand a given number of bits to the right
- The bit positions opened up by the shift are filled by the least significant bits of the source operand
- The source operand is not affected

Shift count of 1:

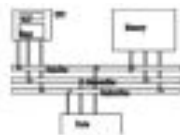
```
mov al,11000001b
```

```
mov bl,00011101b
```

```
shrd al,bl,1
```



SHRD example



Shift AX 4 bits to the right and replace its highest 4 bits with the low 4 bits of DX:

```
mov  ax,234Bh  
mov  dx,7654h  
shrd ax,dx,4
```

Before:



After:



Shift and rotate applications

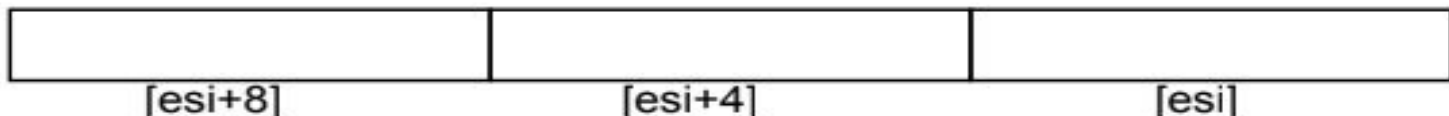
- Shifting Multiple Doublewords
- Binary Multiplication
- Displaying Binary Bits
- Isolating a Bit String



Shifting multiple doublewords

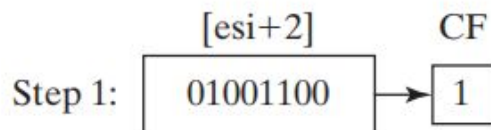
- Programs sometimes need to shift all bits within an array, as one might when moving a bitmapped graphic image from one screen location to another.
- The following shifts an array of 3 doublewords 1 bit to the right:

```
shr array[esi + 8],1 ; high dword  
rcr array[esi + 4],1 ; middle dword,  
rcr array[esi],1      ; low dword,
```

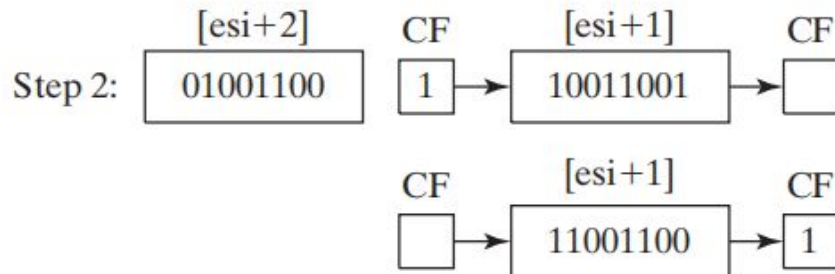


The following steps show how to shift array of bytes one bit to the right:

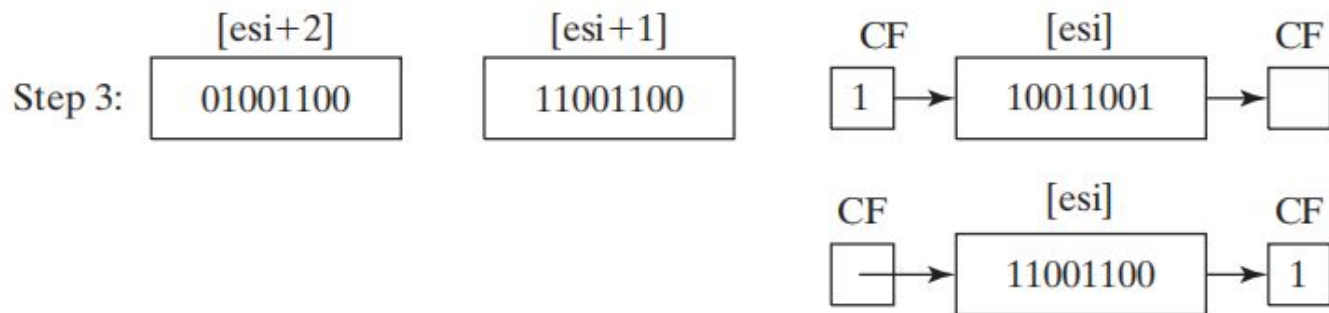
Step 1: Shift the highest byte at [ESI+2] to the right, automatically copying its lowest bit into the Carry flag.



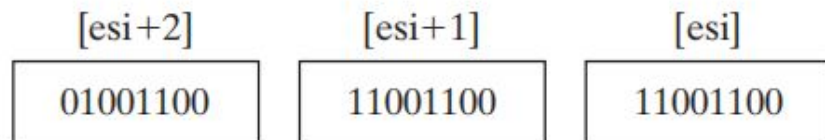
Step 2: Rotate the value at [ESI+1] to the right, filling the highest bit with the value of the Carry flag, and shifting the lowest bit into the Carry flag:



Step 3: Rotate the value at [ESI] to the right, filling the highest bit with the value of the Carry flag, and shifting the lowest bit into the Carry flag:



When finished, all bits have been shifted 1 position to the right:



Example

```
.data
ArraySize = 3
array BYTE ArraySize DUP(99h)    ; 1001 pattern in each nybble
.code
main PROC
    mov esi,0
    shr array[esi+2],1             ; high byte
    rcr array[esi+1],1             ; middle byte, include Carry flag
    rcr array[esi],1              ; low byte, include Carry flag
```



Binary multiplication

- We already know that SHL performs unsigned multiplication efficiently when the multiplier is a power of 2.
- Factor any binary number into powers of 2.
 - For example, to multiply $EAX * 36$, factor 36 into $32 + 4$ and use the distributive property of multiplication to carry out the operation:

36 is **not** a power of two.

But it can be broken down as a **sum of powers of 2**:

$$36 = 32 + 4 = 2^5 + 2^2$$

Then:

$$x \times 36 = x \times (2^5 + 2^2) = (x \times 2^5) + (x \times 2^2)$$

So we can perform:

1. Shift left by 5 (multiply by 32)
2. Shift left by 2 (multiply by 4)
3. Add the two results

$$\text{EAX} * 36$$

$$= \text{EAX} * (32 + 4)$$

$$= (\text{EAX} * 32) + (\text{EAX} * 4)$$

```
; Multiply unsigned EAX by 36 using shifts and adds
mov ebx, eax          ; copy original value
shl eax, 5             ; eax = eax * 32 (2^5)
shl ebx, 2             ; ebx = ebx * 4 (2^2)
add eax, ebx           ; eax = eax*32 + ebx*4 = eax*36
```

The following figure shows the multiplication $123 * 36$, producing 4428 , the product:

		01111011	123
	×	00100100	36

		01111011	123 SHL 2
+		01111011	123 SHL 5

		0001000101001100	4428

It is rather remarkable to discover that bits 2 and 5 are set in the multiplier (36), and the integers 2 and 5 are also the required shift counters. Using this information, the following code excerpt multiplies 123 by 36, using SHL and ADD instructions:

```
.code
mov  eax,123
mov  ebx,eax
shl  eax,5           ; mult by 25
shl  ebx,2           ; mult by 22
add  eax,ebx         ; add the products
```

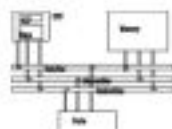

Displaying binary bits



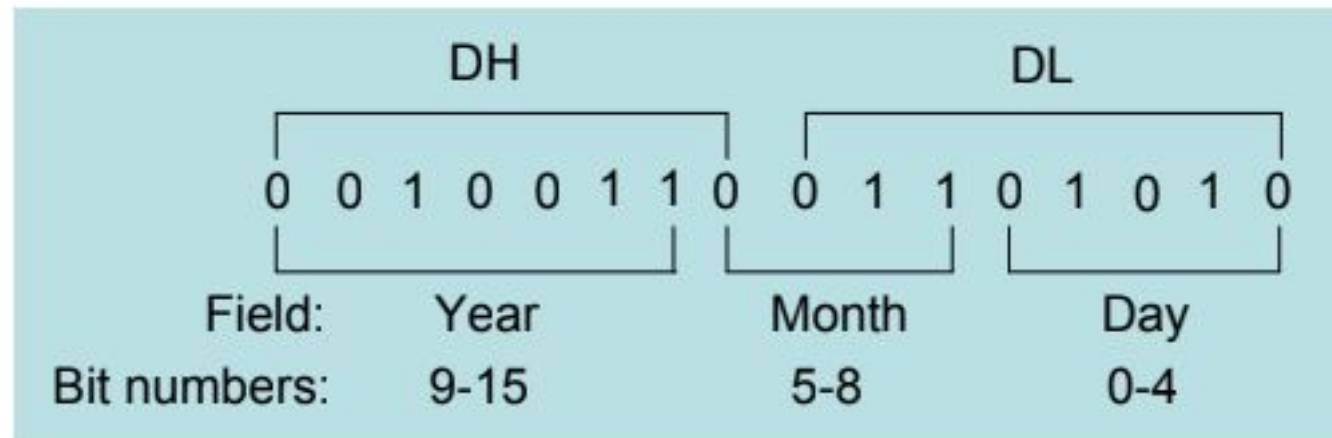
Algorithm: Shift MSB into the Carry flag; If CF = 1, append a "1" character to a string; otherwise, append a "0" character. Repeat in a loop, 32 times.

```
        mov ecx,32
        mov esi,offset buffer
L1:     shl eax,1
        mov BYTE PTR [esi],'0'
        jnc L2
        mov BYTE PTR [esi],'1'
L2:     inc esi
        loop L1
```


Isolating a bit string



- The MS-DOS file date field packs the year (relative to 1980), month, and day into 16 bits:





Isolating a bit string

```
mov al,dl           ; make a copy of DL
and al,00011111b    ; clear bits 5-7
mov day,al          ; save in day variable
```

```
mov ax,dx           ; make a copy of DX
shr ax,5            ; shift right 5 bits
and al,00001111b    ; clear bits 4-7
mov month,al        ; save in month variable
```

```
mov al,dh           ; make a copy of DX
shr al,1            ; shift right 1 bit
mov ah,0            ; clear AH to 0
add ax,1980          ; year is relative to 1980
mov year,ax          ; save in year
```

MUL instruction



- The MUL (unsigned multiply) instruction multiplies an 8-, 16-, or 32-bit operand by either AL, AX, or EAX.
- The instruction formats are:

MUL *r/m8*

MUL *r/m16*

MUL *r/m32*

Implied operands:

Multiplicand	Multiplier	Product
AL	<i>r/m8</i>	AX
AX	<i>r/m16</i>	DX:AX
EAX	<i>r/m32</i>	EDX:EAX

MUL examples



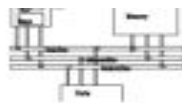
```
mov  al,5h ; AX = 0050h, CF = 0
mov  bl,10h
mul  bl
```

AL		BL		AX	CF
05	×	10	→	0050	0

```
mov  eax,12345h
mov  ebx,1000h
mul  ebx ; EDX:EAX = 0000000012345000h, CF = 0
```

AL		BX		EDX	EAX	CF
00012345	×	00001000	→	00000000	12345000	0

MUL examples



100h * 2000h, using 16-bit operands:

```
.data
val1 WORD 2000h
val2 WORD 100h
.code
mov ax, val1
mul val2 ;
```

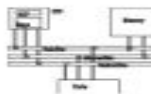
The Carry flag indicates whether or not the upper half of the product contains significant digits.



12345h * 1000h, using 32-bit operands:

```
mov eax, 12345h
mov ebx, 1000h
mul ebx ; EDX:EAX=0000000012345000h, CF=0
```

Example	Operation	Product	Result Registers	CF/OF
AL = 20h, BL = 10h	8-bit × 8-bit	0200h	AX = 0200h → AH ≠ 0	1
AX = 2000h, BX = 0100h	16-bit × 16-bit	0020 0000h	DX = 0020h, AX = 0000h	1
EAX = 20000000 EBX = 00000100h	32-bit × 32-bit	00000020 00000000h	EDX = 00000020h, EAX = 00000000h	1



IMUL instruction

- IMUL (signed integer multiply) multiplies an 8-, 16-, or 32-bit signed operand by either AL, AX, or EAX (there are one/two/three operand format)
- Preserves the sign of the product by sign-extending it into the upper half of the destination register

Example: multiply $48 * 4$, using 8-bit operands:

```
mov    al,48
mov    bl,4
imul   bl           ; AX = 00C0h, OF=1
```

OF=1 because AH is not a sign extension of AL.

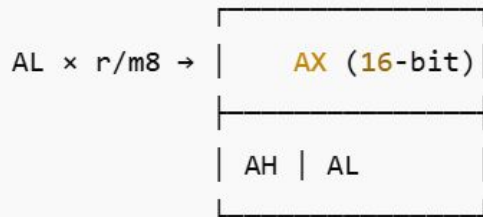
AL = C0h (1100 0000b) → negative number (since MSB=1)

AH = 00h (0000 0000b) → positive sign bits (MSB=0)

The following instructions multiply 48 by 4, producing +192 in DX:AX. DX is a sign extension of AX, so the Overflow flag is clear:

```
mov    ax, 48
mov    bx, 4
imul   bx                ; DX:AX = 000000C0h, OF = 0
```

```
mov    eax, +4823424
mov    ebx, -423
imul   ebx                ; EDX:EAX = FFFFFFFF86635D80h, OF = 0
```



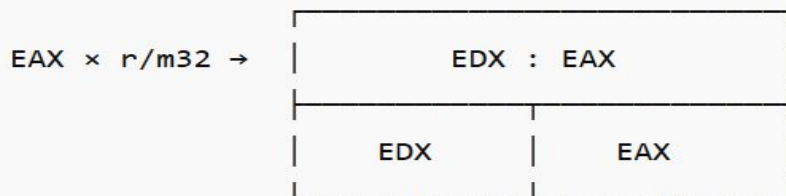
Upper 8 ← Lower 8 (Result in AL)

- For **MUL** : CF/OF = 1 if AH $\neq$ 00h
- For **IMUL** : CF/OF = 1 if AH $\neq$ sign extension of AL



Upper 16 ← Lower 16 (Result in AX)

- For **MUL** : CF/OF = 1 if DX $\neq$ 0000h
- For **IMUL** : CF/OF = 1 if DX $\neq$ sign extension of AX



Upper 32 ← Lower 32 (Result in EAX)

- For **MUL** : CF/OF = 1 if EDX $\neq$ 0
- For **IMUL** : CF/OF = 1 if EDX $\neq$ sign extension of EAX

DIV instruction



- The DIV (unsigned divide) instruction performs 8-bit, 16-bit, and 32-bit division on unsigned integers
- A single operand is supplied (register or memory operand), which is assumed to be the divisor
- Instruction formats:

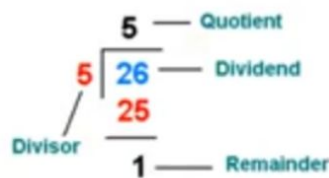
DIV *r/m8*

DIV *r/m16*

DIV *r/m32*

Default Operands:

Dividend	Divisor	Quotient	Remainder
AX	<i>r/m8</i>	AL	AH
DX:AX	<i>r/m16</i>	AX	DX
EDX:EAX	<i>r/m32</i>	EAX	EDX

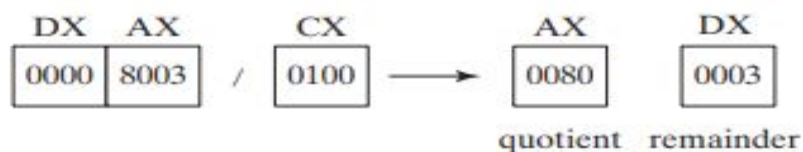


DIV examples



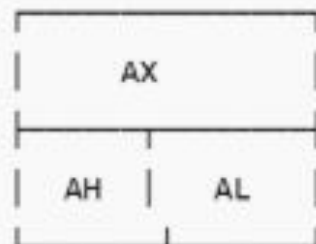
Divide 8003h by 100h, using 16-bit operands:

```
mov dx,0           ; clear dividend, high
mov ax,8003h        ; dividend, low
mov cx,100h         ; divisor
div cx              ; AX = 0080h, DX = 3
```



Same division, using 32-bit operands:

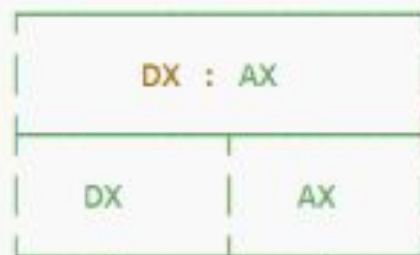
```
mov edx,0           ; clear dividend, high
mov eax,8003h        ; dividend, low
mov ecx,100h         ; divisor
div ecx              ; EAX=00000080h, EDX=3
```



← 16-bit dividend

Remainder
Quotient = AL

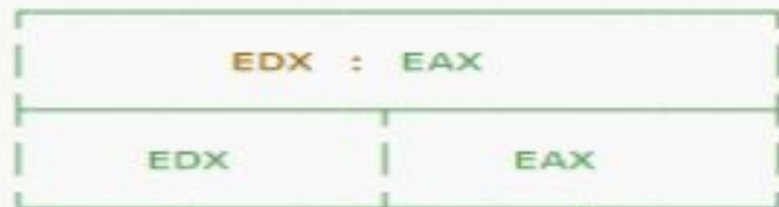
Dividend →



(32-bit)

Remainder
Quotient = AX

Dividend →



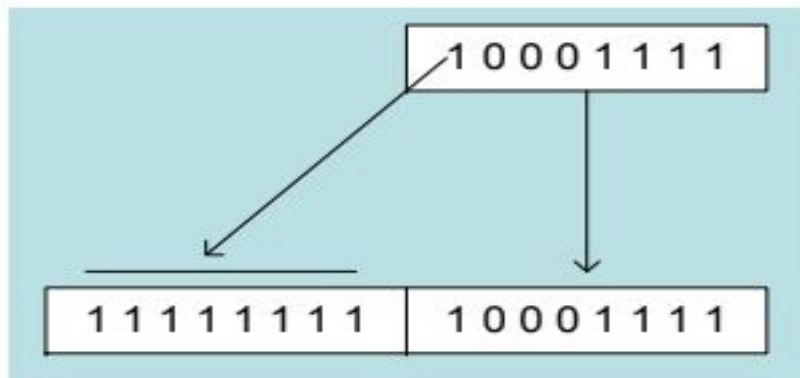
(64-bit)

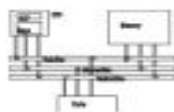
Remainder
Quotient = EAX



Signed integer division

- Signed integers must be sign-extended before division takes place
 - fill high byte/word/doubleword with a copy of the low byte/word/doubleword's sign bit
- For example, the high byte contains a copy of the sign bit from the low byte:

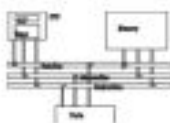




CBW, CWD, CDQ instructions

- The CBW, CWD, and CDQ instructions provide important sign-extension operations:
 - CBW (convert byte to word) extends AL into AH
 - CWD (convert word to doubleword) extends AX into DX
 - CDQ (convert doubleword to quadword) extends EAX into EDX
- For example:

```
mov eax, 0FFFFFF9Bh      ; -101 (32 bits)
cdq                      ; EDX:EAX = FFFFFFFF9Bh
                          ; -101 (64 bits)
```

IDIV instruction

- IDIV (signed divide) performs signed integer division
- Uses same operands as DIV

Example: 8-bit division of -48 by 5

```
mov al,-48
cbw          ; extend AL into AH
mov bl,5
idiv bl      ; AL = -9,  AH = -3
```

IDIV examples



Example: 16-bit division of -48 by 5

```
mov    ax,-48
cwd                    ; extend AX into DX
mov    bx,5
idiv   bx              ; AX = -9,  DX = -3
```

Example: 32-bit division of -48 by 5

```
mov    eax,-48
cdq                    ; extend EAX into EDX
mov    ebx,5
idiv   ebx             ; EAX = -9,  EDX = -3
```

Divide overflow



- ***Divide overflow*** happens when the quotient is too large to fit into the destination.

```
mov ax, 1000h
mov bl, 10h          ; AL cannot hold 100h
div bl
```

It causes a CPU interrupt and halts the program. (divided by zero cause similar results)

Use a 32-bit divisor and 64-bit dividend to reduce the probability of a divide overflow condition. In the following code, the divisor is EBX, and the dividend is placed in the 64-bit combined EDX and EAX registers:

```
mov eax, 1000h
cdq
mov ebx, 10h
div ebx              ; EAX = 00000100h
```

Implementing arithmetic expressions



- Some good reasons to learn how to implement expressions:
 - Learn how compilers do it
 - Test your understanding of MUL, IMUL, DIV, and IDIV
 - Check for 32-bit overflow

Example: `var4 = (var1 + var2) * var3`

```
mov eax,var1
add eax,var2
mul var3
jo  TooBig      ; check for overflow
mov var4,eax    ; save product
```

Implementing arithmetic expressions



Example: $\text{eax} = (-\text{var1} * \text{var2}) + \text{var3}$

```
mov eax,var1
neg eax
mul var2
jo  TooBig      ; check for overflow
add eax,var3
```

Example: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
mov eax,var1      ; left side
mov ebx,5
mul ebx           ; EDX:EAX = product
mov ebx,var2      ; right side
sub ebx,3
div ebx           ; final division
mov var4,eax
```

Implementing arithmetic expressions

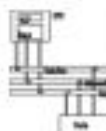


Example: `var4 = (var1 * -5) / (-var2 % var3);`

```
mov    eax,var2        ; begin right side
neg     eax
cdq                     ; sign-extend dividend
idiv   var3            ; EDX = remainder
mov     ebx,edx         ; EBX = right side
mov     eax,-5          ; begin left side
imul    var1           ; EDX:EAX = left side
idiv    ebx            ; final division
mov     var4,eax       ; quotient
```

Sometimes it's easiest to calculate the right-hand term of an expression first.

Exercise . . .



Implement the following expression using signed 32-bit integers:

$$\text{eax} = (\text{ebx} * 20) / \text{ecx}$$

```
mov eax,20  
mul ebx  
div ecx
```



Exercise . . .

Implement the following expression using signed 32-bit integers. Save and restore ECX and EDX:

$$eax = (ecx * edx) / eax$$

```
push ecx
push edx
push eax                ; EAX needed later
mov  eax,ecx
mul  edx                ; left side: EDX:EAX
pop  ecx                ; saved value of EAX
div  ecx                ; EAX = quotient
pop  edx                ; restore EDX, ECX
pop  ecx
```


Exercise . . .



Implement the following expression using signed 32-bit integers. Do not modify any variables other than var3:

$$\text{var3} = (\text{var1} * -\text{var2}) / (\text{var3} - \text{ebx})$$

```
mov  eax,var1
mov  edx,var2
neg  edx
mul  edx          ; left side: edx:eax
mov  ecx,var3
sub  ecx,ebx
div  ecx          ; eax = quotient
mov  var3,eax
```

Extended addition and subtraction

ADC Instruction

- The ADC (add with carry) instruction adds both a source operand and the contents of the Carry flag to a destination operand.
- The instruction formats are the same as for the ADD instruction, and the operands must be the same size:

ADC *reg, reg*

ADC *mem, reg*

ADC *reg, mem*

ADC *mem, imm*

ADC *reg, imm*

`mov dl, 0`

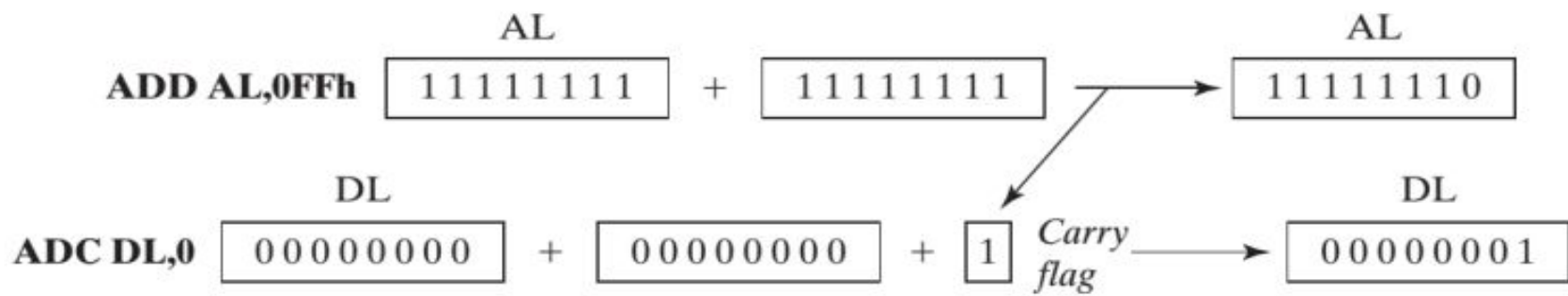
`mov al, 0FFh`

`add al, 0FFh`

`adc dl, 0`

`; AL = FEh`

`; DL/AL = 01FEh`





ADC instruction

- ADC (add with carry) instruction adds both a source operand and the contents of the Carry flag to a destination operand.
- Example: Add two 32-bit integers (FFFFFFFFh + FFFFFFFFFh), producing a 64-bit sum:

```
mov  edx, 0
mov  eax, 0FFFFFFFFh
add  eax, 0FFFFFFFFh
adc  edx, 0 ;EDX:EAX = 00000001FFFFFFFFh
```

Extended addition example



- Add two integers of any size
- Pass pointers to the addends (ESI, EDI) and sum (EBX), ECX indicates the number of doublewords

L1:

```
mov eax,[esi] ; get the first integer
adc eax,[edi] ; add the second integer
pushfd        ; save the Carry flag
mov [ebx],eax ; store partial sum
add esi,4     ; advance all 3 pointers
add edi,4
add ebx,4
popfd         ; restore the Carry flag
loop L1       ; repeat the loop
adc word ptr [ebx],0 ; add leftover carry
```



Extended addition example

```
.data
op1 QWORD 0A2B2A40674981234h
op2 QWORD 08010870000234502h
sum DWORD 3 dup(?)
      ; = 0000000122C32B0674BB5736

.code
...
mov  esi,OFFSET op1 ; first operand
mov  edi,OFFSET op2 ; second operand
mov  ebx,OFFSET sum ; sum operand
mov  ecx,2           ; number of doublewords
call Extended_Add
...
```



SBB instruction

- The SBB (subtract with borrow) instruction subtracts both a source operand and the value of the Carry flag from a destination operand.
- The following example code performs 64-bit subtraction. It sets EDX:EAX to 00000000100000000h and subtracts 1 from this value. The lower 32 bits are subtracted first, setting the Carry flag. Then the upper 32 bits are subtracted, including the Carry flag:

```
mov  edx,1           ; upper half
mov  eax,0           ; lower half
sub  eax,1           ; subtract 1
sbb  edx,0           ; subtract upper half
```

mov	edx, 7	; upper half
mov	eax, 1	; lower half
sub	eax, 2	; subtract 2
sbb	edx, 0	; subtract upper half

